

ARTIFICIAL INTELLIGENCE

Assignment 2

Giannis Nikolentzos & Andreas Doumouras

Thursday, May 29, 2025

This handout includes **coding tasks** and **questions**. Before the deadline, you should submit here a **.zip** file (**not** .rar file) named `lastname.firstname.zip` containing a `/code/` folder (itself containing your scripts) and a report containing your answers to the questions (a .pdf file) or just a jupyter notebook (.ipynb file) that contains both the code and the answers to the questions. The code can be written either in Python or in Java. You can download an .ipynb file from Google Colab by clicking File → Download → Download .ipynb. Your answers should be well constructed and well justified. They should not repeat the question or generalities in the handout. When relevant, you are welcome to include figures, equations and tables derived from your own computations, theoretical proofs or qualitative explanations. **One submission is required for each team.** Each team can consist of up to 2 persons. **The deadline for this assignment is June 11, 2025 11:59 PM.** No extension will be granted. Late submissions will not be graded. Note that the assignment is **mandatory**. If you do not submit the assignment or if you submit it and your grade is less than 5/20, then you have **failed the course**.

1 Introduction

In this assignment, you will represent textual documents as vectors and then use machine learning algorithms to classify those documents. Specifically, your answers to the questions of Assignment 1 will serve as our collection of documents. Therefore, each document belongs to one out of four classes (i.e., the four different questions). There are 148 documents in total which are stored in the `assignment1_dataset.txt` file. Each row of the file contains the class label of a document and its textual content. The following string is used to separate the class label from the textual content: `||`.

You will build a machine learning system that, given an answer to one of the four questions, it will be able to predict to which question this answer corresponds. We will treat the problem as a classification one, where the goal is to predict the class label for a given answer. In what follows, we use the word “document” to refer to an answer. The pipeline that is typically followed to deal with the problem is similar to the one applied in any classification problem; the goal is to train a classifier on a collection of training documents (i.e., those documents whose class labels are known) and then to predict the class labels of unlabeled documents.

You can find in the following link a jupyter notebook with blanks to fill in: https://colab.research.google.com/drive/1fSk1PM9Lp680bRxYU2jgGbVD_ozJSn58?usp=sharing

2 Bag-of-words Representation

Standard machine learning algorithms (e.g., logistic regression, k -nn algorithm, decision trees, etc.) operate on data represented as vectors. Therefore, to classify textual documents, we first need to transform each document, which is typically a string of characters, into a vector. Here, we will employ the *bag-of-words* representation. Specifically, each document is represented by a vector in an n -dimensional space, where each dimension corresponds to a term (i.e., word) from the overall vocabulary of the given document collection. More formally, let $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$ denote a collection of m documents, and $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ be the vocabulary, i.e., the set of words that occur in the documents. $\mathcal{T}$ is typically obtained by applying some standard natural language processing techniques, such as removal of punctuation marks and tokenization. Each document $d_i \in \mathcal{D}$ is represented as a vector of term weights $\mathbf{x}_i = (w_{i1}, w_{i2}, \dots, w_{in})^\top \in \mathbb{R}^n$, where w_{ij} is the weight of term j in document d_i . That way, data can be represented by the *Document-Term matrix* $\mathbf{X} \in \mathbb{R}^{m \times n}$, where the rows correspond to documents and the columns to the different terms (i.e., features) of set $\mathcal{T}$. The Document-Term matrix is illustrated in Figure 1 below. In our implementation, the weight w_{ij} will be set equal to the frequency of term j in

		Terms			
		t_1	t_2	t_j	t_n
Documents	d_1				
	d_2				
	d_i			w_{ij}	
	d_m				

Figure 1: Illustration of the Document-Term matrix.

document d_i , or equal to 0 if term j does not appear in document d_i . For example, given two documents $d_1 = \text{"John loves Mary"}$ and $d_2 = \text{"To be or not to be"}$, the vocabulary $\mathcal{T}$ consists of the following 7 terms: {be, John, loves, Mary, not, or, to}, and the Document-Term matrix is defined as follows:

$$\mathbf{X} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

Additionally, each document d_i is associated with a class label $y_i \in \{0, 1, 2, 3\}$ (i.e., corresponding to the four different questions) forming the class vector $\mathbf{y}$, and the goal is to predict the class labels for a set of test documents.

Task 1 (1.5 points)

Construct the Document-Term matrix $\mathbf{X}$ where the rows represent the answers to the questions of Assignment 1. To extract the vocabulary, you should convert all text to lowercase, remove punctuation (e.g., commas (,), full stops (.), exclamation marks (!), apostrophes ('), etc.), and split text into words. Set the weight w_{ij} of term j in document d_i equal to the frequency of term j in document d_i (you can use the `CountVectorizer` class of the `scikit-learn` package which directly constructs the Document-Term matrix from the list of raw text. Note that `CountVectorizer` produces a sparse representation of the counts. You can transform it into a standard NumPy matrix using `X.toarray()`).

A matrix is sparse if most of its elements are equal to zero. Otherwise, the matrix is dense. Formally, the sparsity of a matrix M is equal to:

$$\text{sparsity}(M) = \frac{\#\text{elements of } M \text{ that are equal to } 0}{\#\text{elements of } M}$$

Task 2 (0.5 points)

Compute the sparsity of the Document-Term matrix X you constructed.

Question 1 (1 points)

Do you expect the Document-Term matrix to be dense or sparse and why?

3 Distance Between Documents

By representing textual documents as vectors, we can compute how similar two documents are to each other by just calculating their Euclidean distance. The Euclidean distance between the vector representations of two documents $\mathbf{x}_i = (w_{i1}, w_{i2}, \dots, w_{in})^\top$ and $\mathbf{x}_j = (w_{j1}, w_{j2}, \dots, w_{jn})^\top$ is defined as:

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{(w_{i1} - w_{j1})^2 + (w_{i2} - w_{j2})^2 + \dots + (w_{in} - w_{jn})^2}$$

Task 3 (1 points)

Construct a matrix $D \in \mathbb{R}^{148 \times 148}$ where the element in position i, j stores the Euclidean distance between the i -th and j -th document.

Task 4 (1 points)

The diagonal elements of matrix D are equal to 0 since the distance between a vector and itself is 0. Note also that the matrix is symmetric. Find the 5 smallest elements of the upper triangular part (the elements above the diagonal) of D . Those distances correspond to 5 pairs of documents. Print those 5 pairs of documents along with their Euclidean distances.

Question 2 (1 points)

Suppose that a dataset contains m documents and each document is a sequence of words. The length of the sequence can range from 1 word to n words. The same word might occur multiple times, e.g., k times within the sequence. What is the maximum possible Euclidean distance between documents of this dataset if we use the bag-of-words representation of section 2?

Question 3 (1 points)

Let d_1 and d_2 represent the following two documents:

$d_1 = \text{"John loves Mary"}$

$d_2 = \text{"Mary loves John"}$

What is the Euclidean distance between d_1 and d_2 if we use the bag-of-words representation of section 2? Do you think that this value of the Euclidean distance accurately captures the relationship between the two sentences or not, and why?

4 Text Categorization

After having formed the Document-Term matrix, the problem has been transformed to a typical classification task: the rows of the matrix correspond to the instances (i.e., documents) and the columns

to the features (i.e., terms). Recall that, for each document we also have class information. In order to train a classification model, we will split the dataset into two sets: (i) training set; and (ii) test set. The training set will be used to learn the parameters of the classification model, that later will be applied to the test part in order to determine the performance of the model.

Task 5 (0.5 points)

Split the dataset (Document-Term matrix and vector of class labels) into two sets: (i) training set; and (ii) test set. The training set will consist of the first 120 documents and the test set of the remaining 28 documents.

The next step of the pipeline involves the selection of the appropriate learning (i.e., classification) algorithm for the problem.

4.1 k -nn Algorithm

We will first use the k -nearest neighbors (k -nn) algorithm to classify the documents. If d_i is a document of the test set that we would like to classify, and $\{d_1, d_2, \dots, d_m\}$ are the documents of the training set, then k -nn performs the following steps:

1. It computes the Euclidean distance between the document d_i and all training documents $\{d_1, d_2, \dots, d_m\}$. That is, it computes the distances $\|\mathbf{x}_i - \mathbf{x}_1\|_2, \|\mathbf{x}_i - \mathbf{x}_2\|_2, \dots, \|\mathbf{x}_i - \mathbf{x}_m\|_2$. Note that all those distances are stored in a single row of matrix $\mathbf{D}$ (the one that corresponds to the test sample).
2. It sorts the distances and finds the k nearest neighbors of document d .
3. Each of the k neighbors “votes” for their class. The class with the most votes is assigned to document d . In case of a tie between two or more classes, one of the classes is chosen at random.

Task 6 (3 points)

Implement the k -nn algorithm. Set $k = 3$ and use the algorithm to make predictions for the samples of the test set. Note: do not use an available implementation of the k -nn algorithm. Implement the algorithm from scratch.

For the evaluation of the classification results, we will compute the *accuracy*. The accuracy is defined as:

$$\text{Accuracy} = \frac{\text{\#correct classifications}}{\text{\#classifications}}$$

For example, if the classifier predicted the following class labels for 5 samples: 0, 3, 1, 0, 2 and the actual (ground-truth) labels are: 0, 3, 1, 0, 3, then the accuracy is equal to $4/5 = 0.8$.

Task 7 (0.5 points)

Compute the accuracy of the 3-nn algorithm (you can use the `accuracy_score()` function of scikit-learn).

Task 8 (0.5 points)

Visualize the performance of the k -nn algorithm as a function of k (you can use the `plot()` function of the Matplotlib package).

Question 4 (2 points)

Suppose that we multiplied the weight of the terms “ $\kappa\alpha\iota$ ” (the term “and” in greek) by 1000. That is, if the term “ $\kappa\alpha\iota$ ” appears twice in a document, its weight will be set equal to 2000 instead of 2. The weights of the rest of the terms are not modified. Could that have any impact on the performance of the k -nn algorithm and why?

4.2 Logistic Regression

Next, you will experiment with the logistic regression classifier. Given a document d represented as a vector $\mathbf{x} = (w_1, w_2, \dots, w_n)^\top$, the multiclass logistic regression algorithm can be used to classify the document into one of the four different classes. It first computes the logits as follows:

$$\mathbf{z} = \mathbf{W} \mathbf{x} + \mathbf{b}$$

where $\mathbf{W} \in \mathbb{R}^{4 \times n}$ is a matrix of learnable parameters and $\mathbf{b} \in \mathbb{R}^4$ is a learnable bias vector. Then the softmax function is applied pointwise to vector $\mathbf{z} \in \mathbb{R}^4$ to obtain probabilities across the four classes. The probability that the document belongs to the j -th class is equal to:

$$\hat{y}_j = \text{softmax}(\mathbf{z})_j = \frac{e^{\mathbf{z}_j}}{\sum_k e^{\mathbf{z}_k}}$$

In the example of section 2, the document “*To be or not to be*” was represented as the following vector:

$$\mathbf{x} = \begin{bmatrix} 2 & 0 & 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

If $\mathbf{W}$ and $\mathbf{b}$ were defined as follows:

$$\mathbf{W} = \begin{bmatrix} 1.2 & 2.5 & -1.7 & 3.6 & 1.4 & 0.1 & 1.3 \\ -1.0 & -2.1 & -1.2 & 0.2 & -1.2 & 0.3 & 1.5 \\ 2.1 & -3.2 & 0.8 & 0.1 & 1.8 & 2.6 & 3.1 \\ 0.5 & 0.6 & 4.1 & -0.5 & -0.5 & 0.2 & 0.2 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 0.5 \\ 1.0 \\ 1.1 \\ 0.3 \end{bmatrix}$$

then the logits would be equal to:

$$\mathbf{z} = \mathbf{W} \mathbf{x} + \mathbf{b} = \begin{bmatrix} 7.0 & 1.1 & 15.9 & 1.4 \end{bmatrix}$$

The output of the softmax would then be:

$$\text{softmax}(\mathbf{z}) = \begin{bmatrix} 1.3637e-04 & 3.7358e-07 & 9.9986e-01 & 5.0428e-07 \end{bmatrix}$$

The algorithm would thus predict that the sample belongs to class 3 since the element in the third position has the highest probability (≈ 0.99).

Task 9 (1.5 points)

Train the logistic regression algorithm on the training set that you have constructed (you can use any available implementation of logistic regression such as the `LogisticRegression` class of `scikit-learn`). Then use the algorithm to make predictions for the documents of the test set. Compute the classification accuracy on the test set.

Question 5 (2 points)

Suppose that we multiplied the weight of the terms “ $\kappa\alpha\iota$ ” (the term “and” in greek) by 1000. That is, if the term “ $\kappa\alpha\iota$ ” appears twice in a document, its weight will be set equal to 2000 instead of 2. The weights of the rest of the terms are not modified. Could that have any impact on the performance of the logistic regression algorithm and why?

Question 6 (1 points)

Suppose that you plan to deploy the logistic regression model of our problem (i.e., classifying the answers to four classes) on a very small mobile device and you can just afford to have 8 columns in the Document-Term matrix $\mathbf{X}$. Which 8 terms would you choose to include into the vocabulary $\mathcal{T}$ and why?

Task 10 (2 points)

Once you have trained the logistic regression classifier, retrieve matrix $\mathbf{W} \in \mathbb{R}^{4 \times n}$ where n is the size of the vocabulary (for an object of the `LogisticRegression` class, matrix $\mathbf{W}$ is stored in the `coef_` attribute of the object). The four rows of $\mathbf{W}$ correspond to the four classes. For each row (i.e., class), find which column has the highest value. Then, find and print the terms that correspond to those columns (the terms that correspond to the n columns are stored in the `vocabulary_` attribute of the `CountVectorizer` object).